Assistance matching system backend [redacted]

Act as a software engineer and write backend code only for this system based on the given requirements; all the code must be in one class. The language to be used is Python, and make sure the code is simple and reliable.
user 1 = needs assistant
user 2 = helper
1- The app shall contain an option for requesting assistance and an option for offering assistance.
1- The user 1 shall be able to submit a help request through the app in which they can indicate where they are, where they want to go and the time they need help.
2- The user 2 shall be able to see help requests and respond to them.
3- The system shall handle request conflicts in a way that there is always only one helper who accepts the request.
4- The user 2 shall receive only the necessary information to help user 1 safely through the app.
5- User 1 and user 2 shall be able to communicate safely through the app.
6- The system shall handle high traffik.
7- the system shall protect user data.

Show less

Check frontend design skill for reference >

Here's the backend implementation. I'll build it as a single class with all requirements addressed — atomic conflict resolution, anonymized messaging, rate limiting, and data protection baked in.

Created a file, ran a command >

All checks pass. Here's a breakdown of every decision made:

Presented file >

Architecture & Design Decisions

Requirement → Implementation mapping

Req	Solution	Why
Submit request (location + time)	`submit_request()`	Validates input, encrypts locations before INSERT
Helper sees & responds to requests	`list_open_requests()` + `accept_request()`	Two-step: browse (no PII) → accept (PII unlocked)
Only one helper accepts (req #3)	`BEGIN EXCLUSIVE` transaction + `WHERE status='open'` guard	SQLite exclusive lock = zero-cost distributed mutex. Second helper gets `ValueError` immediately
Helper gets minimum necessary info (req #4)	`list_open_requests()` returns **no location** — only `request_id`, `needed_at`, `created_at`	Location decrypted and returned **only** in `accept_request()`
Safe communication (req #5)	`send_message()` / `get_messages()` store `sender_role` ("requester"/"helper") not user ID or name	Neither party ever sees the other's identity through the message API
High traffic (req #6)	WAL mode on SQLite, per-thread connections, sliding-window rate limiter (30 req/60s per user)	WAL allows concurrent readers; rate limiter prevents flooding/DoS
Data protection (req #7)	Passwords: `bcrypt` hashed · Locations & messages: **AES-256-GCM encrypted at rest** · Auth: HMAC-SHA256 JWTs with expiry	Defense in depth — even if DB is leaked, PII is unreadable

Key security properties

- **Timing-safe login** — dummy bcrypt check when username doesn't exist prevents username enumeration.
- **PII isolation** — `sender_id` is stored in the DB for audit, but the public `get_messages()` API returns only `sender_role`, keeping both parties anonymous to each other.
- **Token expiry** — JWTs expire in 1 hour; no server-side session state needed.

Production upgrade path

When you outgrow SQLite, swap `_db()` for a **PostgreSQL** connection pool (e.g. `psycopg2` + `pg_pool`) and replace `BEGIN EXCLUSIVE` with `SELECT ... FOR UPDATE SKIP LOCKED` — the rest of the class is unchanged.

I want you to make it shorter and simpler (max 250 line of code)

Ran 2 commands, viewed a file, created a file >

217 lines — all checks pass.

Presented file >

Here's what changed to get it under 250 lines without losing anything critical:

Removed — verbose docstrings, redundant comments, the status constants, the `_get_user()` helper, `get_request_status()` (status is returned by every mutating call), and the `@contextmanager` wrapper (replaced by a plain `_conn()` getter).

Kept — every security property: bcrypt hashing with timing-safe dummy check, AES-GCM encryption for locations and messages, HMAC-SHA256 JWT with expiry, atomic `BEGIN EXCLUSIVE` for conflict-free acceptance, rate limiter, and the privacy rule that helpers never see location data until they accept.

Trade-off made — the in-memory SQLite connection is now per-instance rather than per-thread with a shared file. For a real deployment you'd pass a real `db_path` and each thread gets its own WAL-mode connection to the same file — that one line is the only change needed.